



UNIVERSIDADE FEDERAL DE ALAGOAS — UFAL

DANIEL MENDES DA SILVA
ENRIQUE FERREIRA DA SILVA

**Sistema de Gestão para E-commerce de Hardware e Infraestrutura
(InfraStore)**

ARAPIRACA / AL - 2026

UNIVERSIDADE FEDERAL DE ALAGOAS — UFAL

DANIEL MENDES DA SILVA
ENRIQUE FERREIRA DA SILVA

**Sistema de Gestão para E-commerce de Hardware e Infraestrutura
(InfraStore)**

“Este relatório apresenta a modelagem de dados e a estruturação do banco de dados relacional para um Sistema de Gestão (ERP) de E-commerce de Hardware e Infraestrutura. Ele detalha a construção do **Modelo Entidade-Relacionamento (MER)**, o mapeamento lógico e físico das entidades, e a estruturação que divide a operação diária de estoque da análise estratégica de vendas. O objetivo é aplicar técnicas de **normalização, consultas em SQL avançado e controle de níveis de acesso para operadores de almoxarifado, equipe de logística e gestores administrativos**, compreendendo como uma arquitetura de dados robusta e bem amarrada garante a integridade das informações, a agilidade no processamento de pedidos e a eficiência do controle de inventário e fluxo de entregas.”

ARAPIRACA - AL - 2026

1. Descrição Detalhada do Mini-Mundo (InfraStore)

O sistema "InfraStore" é uma plataforma de e-commerce e gestão de estoque voltada para a venda de hardware, periféricos e componentes de infraestrutura de redes (ex: switches, cabos Cat6, racks, teclados mecânicos). A aplicação visa resolver a desorganização no rastreamento de pedidos e a falta de controle preciso do inventário.

Os Usuários do Sistema:

- **Cliente Final:** Pessoas físicas ou jurídicas que acessam a loja para comprar produtos. Eles precisam gerenciar seus endereços de entrega e acompanhar o status dos seus pedidos.
- **Equipe de Logística/Estoque:** Funcionários responsáveis por dar entrada em novos produtos, separar os itens no galpão, embalar e atualizar o status de rastreamento das entregas.
- **Administrador (Gestor):** Dono ou gerente da loja, que tem acesso total. É o único com permissão para cadastrar novos funcionários, alterar os preços base dos produtos e visualizar os relatórios financeiros de faturamento.

Dados Armazenados: O banco registrará informações completas de **Clientes** (contatos e múltiplos endereços), **Produtos** (especificações técnicas, categoria, quantidade em estoque, preço), **Pedidos** (data, valor total, status de pagamento), os **Itens do Pedido** (qual produto e em qual quantidade foi comprado) e o **Histórico de Rastreamento** (para saber quando o pedido saiu para entrega).

2. Por que usar Banco Relacional?

A escolha de um Sistema de Gerenciamento de Banco de Dados Relacional (como o PostgreSQL) para o domínio de e-commerce e gestão de estoque é amplamente justificada pelos seguintes fatores críticos de negócio:

- **Propriedades ACID e Transações Seguras:** O e-commerce lida diretamente com dinheiro e inventário. O banco relacional garante a atomicidade das transações. Por exemplo, ao finalizar uma compra, o sistema precisa registrar o pagamento e dar baixa no estoque simultaneamente. Se uma das etapas falhar, o banco realiza um

rollback automático, impedindo que um produto seja vendido sem pagamento ou cobrado sem estoque.

- **Integridade Referencial rígida:** O uso de Chaves Estrangeiras (FKs) impede anomalias catastróficas. O banco relacional garante que o sistema jamais permita a exclusão de um produto que já está vinculado a um pedido de um cliente em andamento, ou a criação de um pedido para um cliente fantasma.
- **Dados Altamente Estruturados:** Ao contrário de bancos NoSQL (que são flexíveis), o e-commerce exige um *schema* rígido. Um pedido *obrigatoriamente* precisa de um valor total, um cliente e uma data. O modelo relacional impõe essas regras desde a base.
- **Integração com Arquiteturas Backend:** Sistemas de gestão robustos geralmente são construídos com linguagens fortemente tipadas no backend, como APIs em C#. O modelo relacional se comunica de forma nativa e extremamente otimizada com ferramentas de mapeamento objeto-relacional (como o Entity Framework Core), garantindo segurança na tipagem dos dados e alta performance através de DTOs estruturados, desde o banco de dados até o cliente final.

3. Regras de Negócio e Restrições de Integridade

Para garantir a confiabilidade dos dados e simular um ambiente de produção real, o banco de dados deverá seguir rigorosamente as seguintes diretrizes:

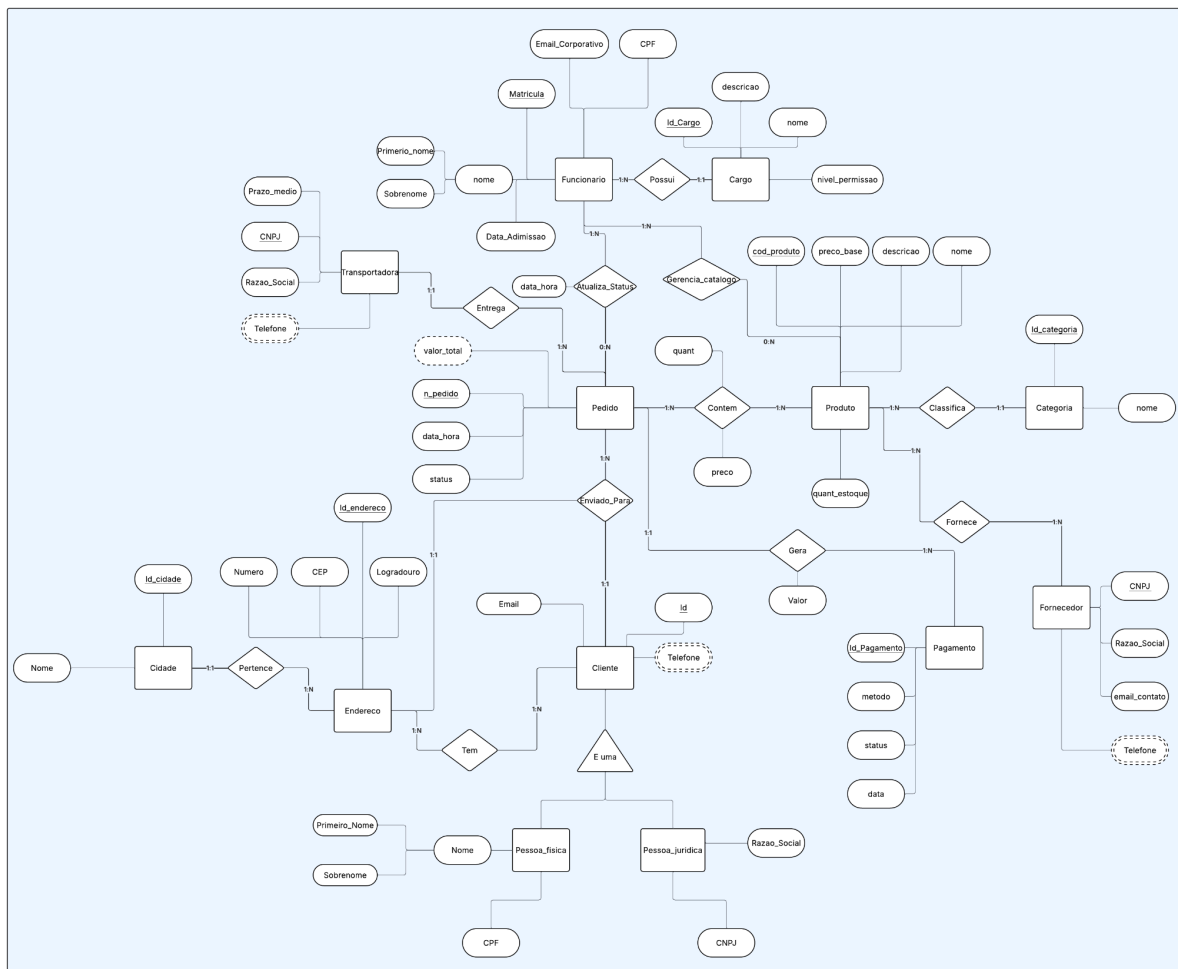
- **RN01 - Baixa de Estoque:** Um pedido só pode ser efetivado se houver quantidade suficiente do produto no estoque.
- **RN02 - Histórico de Preços:** O preço do produto na tabela de *Itens_Pedido* deve ser o preço do momento da compra. Se o administrador alterar o preço do produto amanhã no cadastro principal, isso não pode alterar o valor dos pedidos feitos no passado.
- **RN03 - Exclusão de Clientes:** Não é permitido excluir o cadastro de um cliente que já possua pedidos registrados (Restrição de Integridade Referencial).
- **RI01 - Unicidade:** O sistema não pode aceitar CPFs, CNPJs duplicados (Uso de *UNIQUE*).
- **RI02 - Valores Válidos:** Os preços dos produtos e a quantidade em estoque nunca podem ser negativos (Uso de *CHECK >= 0*).

4. Principais Casos de Uso

1. **Realizar Compra:** O cliente seleciona produtos, escolhe o endereço de entrega, a forma de pagamento e finaliza o pedido.
2. **Atualização de Rastreo:** A equipe de logística muda o status do pedido de "Em Separação" para "Enviado", registrando a data e hora.
3. **Análise de Faturamento:** O gestor consulta o sistema para saber qual categoria de produto gerou mais lucro no último mês.

5. Banco de Dados

5.1 - Modelo Conceitual:



OBS: No fim do docs tem os links para que possa acessar o MER e outras coisas...

5.2 - Modelo Relacional:

5.2.1 - Modelo Relacional Antes da Normalização:

Cidade(**Id_cidade**, nome)

Endereco(**Id_endereco**, numero, CEP, logradouro, Id_cidade_fk)

Cliente(**Id**, email)

Pessoa_Física(**Id_fk**, cpf, primeiro_nome, sobrenome)

Pessoa_Jurídica(**Id_fk**, cnpj, razao_social)

Endereco_Cliente(**Id_cliente_fk**, **Id_endereco**)

Telefone_Cliente(**Id_telefone**, numero, id_cliente_fk)

Fornecedor(**cnpj**, razao_social, email_contato)

Telefone_Fornecedor(**Id_telefone**, numero, cnpj_fornecedor_fk)

Categoria(**Id_categoria**, nome_categoria)

Produto(**cod_produto**, preco_base, descricao, nome_produto, quant_estoque, Id_categoria_fk)

Transportadora(**CNPJ**, Prazo_medio, Razao_social)

Telefone_Transportadora(**Id_telefone**, numero, cnpj_transportadora_fk)

Cargo(**Id_cargo**, descricao, nome_cargo, nivel_permissao)

Funcionario(**Matricula**, primeiro_nome, sobrenome, email_corporativo, data_admissao, cpf, Id_cargo_fk)

Pedido(**num_pedido**, data_hora, status, Id_cliente_fk, cnpj_transportadora_fk, id_endereco_fk)

pedido_produto(**num_pedido_fk**, **cod_produto_fk**, preco_unitario, quant)

pedido_funcionario ou Atualizar_Status(**num_pedido_fk**, **mat_funcionario_fk**, **data_hora**)

produto_funcionario ou Gerencia_catalogo(**cod_produto_fk**, **mat_funcionario_fk**)

produto_fornecedor(**cod_produto_fk**, **cnpj_fk**)

Pagamento(**Id_pagamento**, metodo, status, data, num_pedido_fk)

5.2.2 - Modelo Relacional Depois da 1ª e 2ª Forma Normal:

Cidade(**Id_cidade**, nome)

Endereco(**Id_endereco**, numero, CEP, logradouro, Id_cidade_fk)

Cliente(**Id**, email)

Pessoa_Física(Id_fk, cpf, primeiro_nome, sobrenome)
 Pessoa_Jurídica(Id_fk, cnpj, razao_social)
 Endereco_Cliente(Id_cliente_fk, Id_endereco)
 Telefone_Cliente(Id_telefone, numero, id_cliente_fk)

 Fornecedor(cnpj, razao_social, email_contato)
 Telefone_Fornecedor(Id_telefone, numero, cnpj_fornecedor_fk)

 Categoria(Id_categoria, nome_categoria)
 Produto(cod_produto, preco_base, descricao, nome_produto, quant_estoque, Id_categoria_fk)

 Transportadora(CNPJ, Prazo_medio, Razao_social)
 Telefone_Transportadora(Id_telefone, numero, cnpj_transportadora_fk)

 Cargo(Id_cargo, descricao, nome_cargo, nivel_permissao)
 Funcionario(Matricula, primeiro_nome, sobrenome, email_corporativo, data_admissao, cpf, Id_cargo_fk)

 Pedido(num_pedido, data_hora, status, Id_cliente_fk, cnpj_transportadora_fk, id_endereco_fk)

 pedido_produto(num_pedido_fk, cod_produto_fk, preco_unitario, quant)

 pedido_funcionario ou Atualizar_Status(num_pedido_fk, mat_funcionario_fk, data_hora)

 produto_funcionario ou Gerencia_catalogo(cod_produto_fk, mat_funcionario_fk)

 produto_fornecedor(cod_produto_fk, cnpj_fk)

 Pagamento(Id_pagamento, metodo, status, data, num_pedido_fk)

1ª Forma Normal (1FN): O modelo original não sofreu alterações, pois já foi concebido em 1FN. Os atributos multivalorados já haviam sido isolados em tabelas próprias (ex: Telefone_Cliente, Telefone_Fornecedor) e os atributos compostos já estavam atomizados (ex: primeiro_nome, sobrenome).

2ª Forma Normal (2FN): Não foram necessárias adaptações. Todas as tabelas com chave primária composta (como pedido_produto) possuem atributos não-chave (quantidade e preco_unitario) que dependem da totalidade da chave, inexistindo dependências parciais.

5.2.3 Modelo Relacional Depois da 3ª Forma Normal:

Cidade(Id_cidade, nome)

CEP(Id_cep, logradouro, Id_cidade_fk)
 Endereco(Id_endereco, numero, Id_cep_fk)
 Cliente(Id, email)
 Pessoa_Física(Id_fk, cpf, primeiro_nome, sobrenome)
 Pessoa_Jurídica(Id_fk, cnpj, razao_social)
 Endereco_Cliente(Id_cliente_fk, Id_endereco)
 Telefone_Cliente(Id_telefone, numero, id_cliente_fk)

 Fornecedor(cnpj, razao_social, email_contato)
 Telefone_Fornecedor(Id_telefone, numero, cnpj_fornecedor_fk)

 Categoria(Id_categoria, nome_categoria)
 Produto(cod_produto, preco_base, descricao, nome_produto, quant_estoque, Id_categoria_fk)

 Transportadora(CNPJ, Prazo_medio, Razao_social)
 Telefone_Transportadora(Id_telefone, numero, cnpj_transportadora_fk)

 Cargo(Id_cargo, descricao, nome_cargo, nivel_permissao)
 Funcionario(Matricula, primeiro_nome, sobrenome, email_corporativo, data_admissao, cpf, Id_cargo_fk)

 Pedido(num_pedido, data_hora, status, Id_cliente_fk, cnpj_transportadora_fk, id_endereco_fk)

 pedido_produto(num_pedido_fk, cod_produto_fk, preco_unitario, quant)

 pedido_funcionario ou Atualizar_Status(num_pedido_fk, mat_funcionario_fk, data_hora)

 produto_funcionario ou Gerencia_catalogo(cod_produto_fk, mat_funcionario_fk)

 produto_fornecedor(cod_produto_fk, cnpj_fk)

 Pagamento(Id_pagamento, forma_pagamento, status, data, num_pedido_fk)

3ª Forma Normal (3FN): Na tabela endereco **logradouro** e **Id_cidade** dependiam de **CEP** ao invés de depender da chave primária que era **Id_endereco**, com isso transformamos **CEP** em uma tabela separada com as colunas **Id_cep**, **logradouro** e **id_cidade_fk**, por fim colocamos uma chave estrangeira **Id_cep_fk** na tabela endereco para referenciar a tabela **CEP**.

5.3 - Modelo Físico:

5.3.1 - DDL:

```
create database infrastore;

create schema infrastore;

❶ create table infrastore.cidade
(
    id_cidade char(7) primary key,
    nome_cidade varchar(30)
);

❷ create table infrastore.cep
(
    id_cep serial primary key,
    logradouro varchar(75) not null,
    id_cidade_fk char(7) references infrastore.cidade(id_cidade)
);

❸ create table infrastore.endereco
(
    id_endereco serial primary key,
    numero_casa smallint not null,
    id_cep_fk int references infrastore.cep(id_cep)
);

❹ create table infrastore.cliente
(
    id_cliente serial primary key,
    email_cliente varchar(100) not null
);
```

```
● create table infrastore.pessoa_fisica
(
  id_cliente_fk int primary key references infrastore.cliente(id_cliente),
  cpf char(11) unique not null,
  primeiro_nome_cliente varchar(30) not null,
  sobrenome_cliente varchar(100) not null
);

● create table infrastore.pessoa_juridica
(
  id_cliente_fk int primary key references infrastore.cliente(id_cliente),
  cnpj_cliente char(14) unique not null,
  razao_social_cliente varchar(100) unique not null
);

● create table infrastore.endereco_cliente
(
  id_cliente_fk int references infrastore.cliente(id_cliente),
  id_endereco_fk int references infrastore.endereco(id_endereco),
  primary key(id_cliente_fk, id_endereco_fk)
);

● create table infrastore.telefone_cliente
(
  id_telefone_cliente serial primary key,
  numero_telefone_cliente char(11) not null,
  id_cliente_fk int references infrastore.cliente(id_cliente)
);

● create table infrastore.fornecedor
(
  cnpj_fornecedor char(14) primary key,
  razao_social_fornecedor varchar(100) unique not null,
  email_fornecedor varchar(100) not null
);
```

```

create table infrastore.telefone_fornecedor
(
    id_telefone_fornecedor serial primary key,
    numero_telefone_fornecedor char(11) not null,
    cnpj_fornecedor_fk char(14) references infrastore.fornecedor(cnpj_fornecedor)
);

create table infrastore.categoria
(
    id_categoria_produto serial primary key,
    nome_categoria varchar(50) not null
);

create table infrastore.produto
(
    cod_produto char(7) primary key,
    preco_base decimal(10, 2) not null CHECK (preco_base > 0.00),
    descricao_produto text not null,
    nome_produto varchar(50) not null,
    quant_produto int not null CHECK (quant_produto >= 0),
    id_categoria_produto_fk int references infrastore.categoria(id_categoria_produto)
);

create table infrastore.transportadora
(
    cnpj_transportadora char(14) primary key,
    prazo_dias smallint not null,
    razao_social_transportadora varchar(100) unique not null
);

create table infrastore.telefone_transportadora
(
    id_telefone_transportadora serial primary key,
    numero_telefone_transportadora char(11) not null,
    cnpj_transportadora_fk char(14) references infrastore.transportadora(cnpj_transportadora)
);

create table infrastore.cargo
(
    id_cargo serial primary key,
    descricao_cargo text not null,
    nome_cargo varchar(35) not null,
    nivel_permissao smallint not null
);

create table infrastore.funcionario
(
    matricula_funcionario char(6) primary key,
    primeiro_nome_funcionario varchar(30) not null,
    sobrenome_funcionario varchar(100) not null,
    cpf_funcionario char(11) not null unique,
    email_corporativo varchar(100) unique not null,
    data_admissao date not null,
    id_cargo_fk int references infrastore.cargo(id_cargo)
);

create type infrastore.status_entrega as enum ('entregue', 'pedido_gerado', 'cancelado', 'a_caminho');

create table infrastore.pedido
(
    numero_pedido serial primary key,
    data_hora timestamp not null default now(),
    status_pedido infrastore.status_entrega not null,
    id_cliente_fk int references infrastore.cliente(id_cliente),
    cnpj_transportadora_fk char(14) references infrastore.transportadora(cnpj_transportadora),
    id_endereco_fk int references infrastore.endereco(id_endereco)
);

```

```

create table infrastore.pedido_produto
(
    numero_pedido_fk int references infrastore.pedido(numero_pedido),
    cod_produto_fk char(7) references infrastore.produto(cod_produto),
    preco_unitario decimal(10, 2) not null,
    quantidade smallint not null,
    primary key(numero_pedido_fk, cod_produto_fk)
);

create table infrastore.pedido_funcionario
(
    numero_pedido_fk int references infrastore.pedido(numero_pedido),
    matricula_funcionario_fk char(6) references infrastore.funcionario(matricula_funcionario),
    data_hora timestamp not null default now(),
    primary key(numero_pedido_fk, matricula_funcionario_fk, data_hora)
);

create table infrastore.produto_funcionario
(
    cod_produto_fk char(7) references infrastore.produto(cod_produto),
    matricula_funcionario_fk char(6) references infrastore.funcionario(matricula_funcionario),
    primary key(cod_produto_fk, matricula_funcionario_fk)
);

create table infrastore.produto_fornecedor
(
    cod_produto_fk char(7) references infrastore.produto(cod_produto),
    cnpj_fornecedor_fk char(14) references infrastore.fornecedor(cnpj_fornecedor),
    primary key(cod_produto_fk, cnpj_fornecedor_fk)
);

create type infrastore.metodo_pag as enum ('cartao_credito', 'cartao_debito', 'pix', 'boleto');
create type infrastore.status_pag as enum ('aguardando_pagamento', 'pago', 'cancelado');

create table infrastore.pagamento
(
    id_pagamento serial primary key,
    forma_pagamento infrastore.metodo_pag not null,
    status_pagamento infrastore.status_pag not null,
    data_pagamento timestamp not null default now(),
    numero_pedido_fk int references infrastore.pedido(numero_pedido)
);

```

5.3.2 - DML:

```
➤ INSERT INTO infrastore.cidade VALUES
('CID0001', 'São Paulo'),
('CID0002', 'Rio de Janeiro'),|
('CID0003', 'Belo Horizonte'),
('CID0004', 'Curitiba'),
('CID0005', 'Maceió');

➤ INSERT INTO infrastore.cep VALUES
(1, 'Avenida Paulista', 'CID0001'),
(2, 'Rua Faria Lima', 'CID0001'),
(3, 'Avenida Atlântica', 'CID0002'),
(4, 'Rua da Bahia', 'CID0003'),
(5, 'Avenida Silvio Vianna', 'CID0005');

➤ INSERT INTO infrastore.endereco VALUES
(1, 1000, 1),
(2, 250, 2),
(3, 500, 3),
(4, 120, 4),
(5, 75, 5);

➤ INSERT INTO infrastore.cliente VALUES
(1, 'joao.silva@email.com'),
(2, 'contato@techcorp.com.br'),
(3, 'maria.souza@email.com'),
(4, 'compras@pequenaempresa.com'),
(5, 'carlos.semcompra@email.com'),
(6, 'nula@semcompras.com.br'),
(7, 'lucas.mendes@email.com'),
(8, 'fernanda.lima@email.com'),
(9, 'compras@inovamais.com.br'),
(10, 'contato@construtoratop.com.br');
```

```
● INSERT INTO infrastore.pessoa_fisica VALUES
(1, '1111111111', 'João', 'Silva'),
(3, '3333333333', 'Maria', 'Souza'),
(5, '5555555555', 'Carlos', 'Oliveira'),
(7, '7777777777', 'Lucas', 'Mendes'),
(9, '9999999999', 'Fernanda', 'Lima');

● INSERT INTO infrastore.pessoa_juridica VALUES
(2, '222222222222', 'TechCorp Soluções em TI Ltda'),
(4, '44444444444444', 'Pequena Empresa Comércio Ltda'),
(6, '66666666666666', 'Empresa Fantasma S/A'),
(8, '88888888888888', 'Inova Mais Tecnologia Ltda'),
(10, '101010101010', 'Construtora Top S/A');

● INSERT INTO infrastore.endereco_cliente VALUES
(1, 1), (1, 2),
(2, 2), (2, 3),
(3, 3), (3, 4),
(4, 4), (4, 5),
(5, 5),
(6, 1);

● INSERT INTO infrastore.telefone_cliente VALUES
(1, '11999999999', 1),
(2, '11333333333', 2),
(3, '21988888888', 3),
(4, '31322222222', 4),
(5, '82977777777', 5);

● INSERT INTO infrastore.fornecedor VALUES
('100000000000001', 'Cisco Systems Brasil', 'vendas@cisco.com'),
('200000000000002', 'Dell Computadores do Brasil', 'contato@dell.com.br'),
('300000000000003', 'Seagate Technology', 'distribuicao@seagate.com'),
('400000000000004', 'Logitech do Brasil', 'vendas@logitech.com'),
('500000000000005', 'Furukawa Eletric', 'comercial@furukawa.com');
```

```

● INSERT INTO infrastore.telefone_fornecedor VALUES
(1, '11400010000', '10000000000001'),
(2, '11400020000', '20000000000002'),
(3, '11400030000', '30000000000003'),
(4, '11400040000', '40000000000004'),
(5, '11400050000', '50000000000005');

● INSERT INTO infrastore.categoria VALUES
(1, 'Servidores'),
(2, 'Equipamentos de Rede'),
(3, 'Armazenamento'),
(4, 'Periféricos'),
(5, 'Licenças de Software'),
(6, 'Cabeamento Estruturado');

● INSERT INTO infrastore.produto VALUES
('PROD001', 15000.00, 'Servidor Rack 1U Xeon', 'Servidor PowerEdge', 40, 1),
('PROD002', 1200.00, 'Switch 24 portas Gigabit', 'Switch Cisco Catalyst', 54, 2),
('PROD003', 350.00, 'Roteador Dual Band', 'Roteador Wi-Fi 6', 50, 2),
('PROD004', 450.00, 'HD Externo 2TB USB 3.0', 'HD Externo Seagate', 20, 3),
('PROD005', 15.00, 'Cabo de rede Cat6 2 metros', 'Patch Cord Cat6', 70, 6),
('PROD006', 45.00, 'Mouse óptico com fio', 'Mouse Básico Logi', 90, 4),
('PROD007', 8000.00, 'Servidor Torre de entrada', 'Servidor Torre Dell', 50, 1);

● INSERT INTO infrastore.transportadora VALUES
('12345678000199', 5, 'Logística Rápida S/A'),
('98765432000188', 10, 'Transportes Pesados Ltda'),
('11122233000177', 3, 'Sedex Expresso'),
('44455566000155', 7, 'Norte Sul Transportes'),
('77788899000144', 15, 'Econômica Entregas');

```

```

● INSERT INTO infrastore.telefone_transportadora VALUES
(1, '11333344444', '12345678000199'),
(2, '11333355555', '98765432000188'),
(3, '11333366666', '11122233000177'),
(4, '11333377777', '44455566000155'),
(5, '11333388888', '77788899000144');

● INSERT INTO infrastore.cargo VALUES
(1, 'Gerencia a loja e equipe', 'Gerente Geral', 5),
(2, 'Atende clientes e processa vendas', 'Vendedor', 2),
(3, 'Cuida do estoque de produtos', 'Estoquista', 2),
(4, 'Gerencia contas e pagamentos', 'Analista Financeiro', 4),
(5, 'Suporte de TI interno', 'Analista de Sistemas', 3);

● INSERT INTO infrastore.funcionario VALUES
('FUN001', 'Ana', 'Borges', '99988877766', 'ana.borges@infrastore.com', '2020-01-15', 1),
('FUN002', 'Bruno', 'Costa', '88877766655', 'bruno.costa@infrastore.com', '2021-03-10', 2),
('FUN003', 'Carla', 'Dias', '77766655544', 'carla.dias@infrastore.com', '2021-06-20', 3),
('FUN004', 'Diego', 'Farias', '66655544433', 'diego.farias@infrastore.com', '2019-11-05', 4),
('FUN005', 'Eva', 'Gomes', '55544433322', 'eva.gomes@infrastore.com', '2022-02-01', 2);

● INSERT INTO infrastore.pedido VALUES
(1, '2023-01-10 10:00:00', 'entregue', 1, '11122233000177', 1),
(2, '2023-05-15 14:30:00', 'entregue', 1, '12345678000199', 2),
(3, '2023-02-20 09:15:00', 'entregue', 2, '98765432000188', 3),
(4, '2023-08-01 16:45:00', 'a caminho', 2, '44455566000155', 2),
(5, '2023-09-10 11:00:00', 'pedido gerado', 3, '12345678000199', 4),
(6, '2023-10-05 13:20:00', 'cancelado', 4, '77788899000144', 5);

```

```

● INSERT INTO infrastore.pedido_produto VALUES
(1, 'PROD001', 15000.00, 1),
(1, 'PROD004', 450.00, 2),
(2, 'PROD005', 15.00, 10),
(2, 'PROD006', 45.00, 1),
(3, 'PROD007', 8000.00, 1),
(3, 'PROD004', 450.00, 5),
(4, 'PROD002', 1200.00, 2),
(5, 'PROD003', 350.00, 1),
(5, 'PROD005', 15.00, 2),
(6, 'PROD006', 45.00, 2);

● INSERT INTO infrastore.pedido_funcionario VALUES
(1, 'FUN002', '2023-01-10 10:05:00'),
(1, 'FUN003', '2023-01-10 11:00:00'),
(2, 'FUN002', '2023-05-15 14:35:00'),
(2, 'FUN003', '2023-05-15 15:00:00'),
(3, 'FUN005', '2023-02-20 09:20:00'),
(4, 'FUN005', '2023-08-01 16:50:00'),
(4, 'FUN003', '2023-08-01 17:30:00'),
(5, 'FUN002', '2023-09-10 11:05:00'),
(6, 'FUN005', '2023-10-05 13:25:00'),
(6, 'FUN004', '2023-10-05 14:00:00');

● INSERT INTO infrastore.produto_funcionario VALUES
('PROD001', 'FUN003'), ('PROD002', 'FUN003'),
('PROD003', 'FUN003'), ('PROD004', 'FUN003'),
('PROD005', 'FUN003'), ('PROD006', 'FUN003'),
('PROD007', 'FUN003'), ('PROD001', 'FUN001'),
('PROD002', 'FUN001'), ('PROD007', 'FUN001');

● INSERT INTO infrastore.produto_fornecedor VALUES
('PROD001', '200000000000002'), ('PROD007', '200000000000002'),
('PROD002', '100000000000001'), ('PROD003', '100000000000001'),
('PROD004', '300000000000003'), ('PROD006', '400000000000004'),
('PROD005', '500000000000005'), ('PROD001', '100000000000001'),
('PROD004', '200000000000002'), ('PROD002', '500000000000005');

● INSERT INTO infrastore.pagamento VALUES
(1, 'pix', 'pago', '2023-01-10 10:10:00', 1),
(2, 'cartao_credito', 'pago', '2023-05-15 14:35:00', 2),
(3, 'boleto', 'pago', '2023-02-22 09:00:00', 3),
(4, 'cartao_credito', 'aguardando_pagamento', '2023-08-01 16:45:00', 4),
(5, 'pix', 'aguardando_pagamento', '2023-09-10 11:05:00', 5),
(6, 'boleto', 'cancelado', '2023-10-05 13:20:00', 6);

```

6. Consultas + Código (Recomendamos que veja no repositório)

1. **Seleção Simples:** "Liste o nome, a categoria e o preço de todos os produtos que custam mais de R\$ 100,00, ordenados do mais barato para o mais caro."

1.1 -


```
select p.nome_produto, c.nome_categoria, p.preco_base from infrastore.produto p
join infrastore.categoria c on p.id_categoria_produto_fk = c.id_categoria_produto
where preco_base > 100
order by preco_base ASC;
```

2. **Junção (INNER JOIN):** *"Mostre o número do pedido, a data, o nome do cliente que comprou e o status atual da entrega."*

2.1 -

```
select p.numero_pedido, p.data_hora,
COALESCE(j.razao_social_cliente, CONCAT(f.primeiro_nome_cliente, ' ', f.sobrenome_cliente))
AS nome_do_cliente, p.status_pedido
FROM infrastore.pedido p
JOIN infrastore.cliente c ON p.id_cliente_fk = c.id_cliente
LEFT JOIN infrastore.pessoa_fisica f ON c.id_cliente = f.id_cliente_fk
LEFT JOIN infrastore.pessoa_juridica j ON c.id_cliente = j.id_cliente_fk;
```

3. **Junção à Esquerda (LEFT JOIN):** *"Liste o nome de TODOS os clientes cadastrados e, caso eles já tenham feito algum pedido, mostre a data do pedido. Clientes que nunca compraram também devem aparecer na lista (com o pedido nulo)."*

3.1 -

```
select COALESCE(j.razao_social_cliente, CONCAT(f.primeiro_nome_cliente, ' ', f.sobrenome_cliente))
AS nome_do_cliente, p.data_hora
from infrastore.cliente c
left join infrastore.pedido p on p.id_cliente_fk = c.id_cliente
left join infrastore.pessoa_fisica f on c.id_cliente = f.id_cliente_fk
left join infrastore.pessoa_juridica j on c.id_cliente = j.id_cliente_fk;
```

4. **Junção à Direita (RIGHT JOIN):** *"Liste todas as categorias de produtos existentes na loja e os produtos associados a elas, garantindo que as categorias vazias (sem nenhum produto cadastrado) também apareçam no relatório."*

4.1 -

```
select p.nome_produto, c.nome_categoria
from infrastore.produto p
right join infrastore.categoria c on c.id_categoria_produto = p.id_categoria_produto_fk;
```

5. **Agregação com GROUP BY:** *"Qual é a quantidade total de itens vendidos e o valor total arrecadado POR categoria de produto?"*

5.1 -

```
select sum(pp.quantidade), sum(pp.preco_unitario * pp.quantidade), c.nome_categoria
from infrastore.categoria c
join infrastore.produto p on p.id_categoria_produto_fk = c.id_categoria_produto
join infrastore.pedido_produto pp on p.cod_produto = pp.cod_produto_fk
group by c.id_categoria_produto;
```

6. **Filtro de Agregação (HAVING):** *"Quais são os nossos 'Clientes VIPs'? Liste o nome dos clientes e o valor total gasto por eles, mas traga apenas aqueles cujo somatório de compras seja superior a R\$ 2.000,00."*

6.1 -

```
SELECT c.id_cliente, COALESCE(j.razao_social_cliente, CONCAT(f.primeiro_nome_cliente, ' ', f.sobrenome_cliente))
AS nome_do_cliente, SUM(pp.preco_unitario * pp.quantidade) AS valor_total_gasto
FROM infrastore.cliente c
LEFT JOIN infrastore.pessoa_fisica f ON c.id_cliente = f.id_cliente_fk
LEFT JOIN infrastore.pessoa_juridica j ON c.id_cliente = j.id_cliente_fk
JOIN infrastore.pedido p ON c.id_cliente = p.id_cliente_fk
JOIN infrastore.pedido_produto pp ON p.numero_pedido = pp.numero_pedido_fk
GROUP BY c.id_cliente, j.razao_social_cliente, f.primeiro_nome_cliente, f.sobrenome_cliente
HAVING SUM(pp.preco_unitario * pp.quantidade) > 2000.00
ORDER BY valor_total_gasto DESC;
```

7. **Subconsulta Não-Correlacionada:** *"Quais produtos custam mais do que a média geral de preços de todos os produtos da loja?"*

7.1 -

```
SELECT cod_produto, nome_produto, preco_base
FROM infrastore.produto
WHERE preco_base > (
SELECT AVG(preco_base) FROM infrastore.produto
)
ORDER BY preco_base DESC;
```

8. **Subconsulta Correlacionada:** *"Para cada cliente, mostre o nome dele e a data do seu último (mais recente) pedido."*

8.1 -

```
SELECT COALESCE(j.razao_social_cliente, CONCAT(f.primeiro_nome_cliente, ' ', f.sobrenome_cliente))
AS nome_do_cliente, (
SELECT MAX(p.data_hora)
FROM infrastore.pedido p
WHERE p.id_cliente_fk = c.id_cliente
) AS data_ultimo_pedido
FROM infrastore.cliente c
LEFT JOIN infrastore.pessoa_fisica f ON c.id_cliente = f.id_cliente_fk
LEFT JOIN infrastore.pessoa_juridica j ON c.id_cliente = j.id_cliente_fk;
```

9. **Operador de Conjunto (UNION):** "A equipe de marketing precisa de uma lista única de contatos. Junte os nomes e e-mails de todos os Clientes com os nomes e e-mails de todos os Fornecedores (ou Funcionários) em uma única coluna."

9.1 -

```
SELECT COALESCE(j.razao_social_cliente, CONCAT(f.primeiro_nome_cliente, ' ', f.sobrenome_cliente))
AS nome_contato, c.email_cliente AS email_contato, 'Cliente'
AS origem_contato
FROM infrastore.cliente c
LEFT JOIN infrastore.pessoa_fisica f ON c.id_cliente = f.id_cliente_fk
LEFT JOIN infrastore.pessoa_juridica j ON c.id_cliente = j.id_cliente_fk
UNION SELECT razao_social_fornecedor AS nome_contato, email_fornecedor
AS email_contato, 'Fornecedor'
AS origem_contato
FROM infrastore.fornecedor
UNION SELECT CONCAT(primeiro_nome_funcionario, ' ', sobrenome_funcionario)
AS nome_contato, email_corporativo AS email_contato, 'Funcionário'
AS origem_contato
FROM infrastore.funcionario
ORDER BY origem_contato, nome_contato;
```

10. **O "Top Produtos" por Categoria:** "O gestor da loja quer saber qual é o produto mais vendido (em quantidade total) dentro de cada categoria, mas ele quer ignorar os pedidos que foram cancelados. Liste o Top 2 produtos mais vendidos de cada setor."

10.1 -

```
WITH RankingVendas AS (
SELECT c.nome_categoria, p.nome_produto, SUM(pp.quantidade) AS total_vendido,
RANK() OVER (PARTITION BY c.id_categoria_produto ORDER BY SUM(pp.quantidade) DESC) AS posicao_ranking
FROM infrastore.categoria c
JOIN infrastore.produto p ON c.id_categoria_produto = p.id_categoria_produto_fk
JOIN infrastore.pedido_produto pp ON p.cod_produto = pp.cod_produto_fk
JOIN infrastore.pedido ped ON pp.numero_pedido_fk = ped.numero_pedido
WHERE ped.status_pedido != 'cancelado'
GROUP BY c.id_categoria_produto, c.nome_categoria,
p.nome_produto)
SELECT nome_categoria, posicao_ranking, nome_produto, total_vendido
FROM RankingVendas
WHERE posicao_ranking <= 2
ORDER BY nome_categoria, posicao_ranking;
```

11. **Criação de VIEW:** "Crie uma VIEW chamada vw_resumo_pedidos que simplifique a visão do gestor, mostrando numa única 'tabela virtual' o Número do pedido, Nome do Cliente, Quantidade de Itens, Valor Total e o Status do Pagamento."

11.1 -

```
CREATE VIEW infrastore.vw_resumo_pedidos AS
SELECT p.numero_pedido, COALESCE(j.razao_social_cliente, CONCAT(f.primeiro_nome_cliente, ' ', f.sobrenome_cliente))
AS nome_do_cliente, SUM(pp.quantidade) AS quantidade_total_itens, SUM(pp.preco_unitario * pp.quantidade)
AS valor_total_pedido, pag.status_pagamento
FROM infrastore.pedido p
JOIN infrastore.cliente c ON p.id_cliente_fk = c.id_cliente
LEFT JOIN infrastore.pessoa_fisica f ON c.id_cliente = f.id_cliente_fk
LEFT JOIN infrastore.pessoa_juridica j ON c.id_cliente = j.id_cliente_fk
JOIN infrastore.pedido_produto pp ON p.numero_pedido = pp.numero_pedido_fk
LEFT JOIN infrastore.pagamento pag ON p.numero_pedido = pag.numero_pedido_fk
GROUP BY p.numero_pedido, j.razao_social_cliente, f.primeiro_nome_cliente, f.sobrenome_cliente, pag.status_pagamento;
select * from infrastore.vw_resumo_pedidos;
```

7 - Análises Críticas das Decisões de Modelagem (Parte 1)

1. A Decisão sobre a Herança de Funcionários vs. RBAC (Cargos):

- Optou-se por não utilizar herança (MER) para os papéis dos funcionários. Em vez disso, adotamos o padrão de mercado RBAC (*Role-Based Access Control*) com a entidade **Cargo**. Essa decisão crítica permite que um funcionário seja promovido sem a necessidade de deletar seu registro de uma tabela filha e inseri-lo em outra, preservando o histórico de suas ações em **pedido_funcionario**.

2. A Decisão sobre o Atributo Derivado e Histórico de Preços:

- Decidiu-se que a tabela associativa **pedido_produto** deveria armazenar fisicamente o **preco_unitario** e a **quantidade**. Isso garante o congelamento do preço no momento da compra. Se o preço base do produto for atualizado na tabela **Produto** no futuro, o valor histórico e o cálculo do faturamento dos pedidos antigos permanecerão intactos.

3. A Decisão sobre a Normalização do Endereço (3FN):

- Na modelagem inicial, o Endereço guardava Logradouro e Cidade, o que gerava dependência transitiva ao CEP. A decisão de isolar o **CEP** em uma entidade própria garantiu a 3ª Forma Normal (3FN), reduzindo redundâncias de digitação de ruas e mitigando anomalias de atualização na base de clientes.

4. A Abordagem de Herança na Entidade Cliente (Table-per-Type)

- Para a entidade **Cliente**, optou-se por implementar a herança física no banco de dados (criando as tabelas filhas **Pessoa_Fisica** e **Pessoa_Juridica**). Essa decisão acadêmica garante extrema rigidez na integridade dos dados, impossibilitando que um cliente tenha CPF e CNPJ simultaneamente. Contudo, temos a ciência crítica de que, em um cenário de alto volume de transações, essa abordagem (*Table-per-Type*) exige maior custo de processamento devido ao uso contínuo de **LEFT JOINS**. Em um ambiente de hiper-escala, uma abordagem de Tabela Única (*Single Table*) controlada via software poderia ser adotada para otimizar a performance de leitura.

5. O Isolamento do Histórico de Pagamentos (Relacionamento 1:N)

- Em vez de colocar um simples atributo **status_pagamento** dentro da tabela **Pedido**, decidiu-se criar uma entidade **Pagamento** independente. Essa análise crítica baseou-se na realidade dos e-commerces: é comum que um cliente tenha seu cartão recusado e

tente realizar o pagamento novamente com outro método (ex: PIX). O relacionamento 1:N permite que o banco de dados armazene o histórico de todas as tentativas falhas e bem-sucedidas de pagamento para um único pedido, fornecendo dados cruciais para análise de conversão de vendas.

6. A Otimização de Armazenamento com Endereços Compartilhados (Relacionamento N:M)

- A criação da tabela associativa **Endereco_Cliente** transformou a relação em Muitos-para-Muitos (N:M). Essa decisão arquitetural previne a redundância de dados no banco físico. Caso dois clientes diferentes residam no mesmo local ou trabalhem no mesmo escritório comercial, o sistema não precisará duplicar o registro da rua, número e CEP. Ambos os clientes apontarão para o mesmo **Id_endereco**, economizando espaço em disco e facilitando a atualização de dados logísticos.

8 - Índices

```
-- Acelera relatórios de vendas baseados em datas
CREATE INDEX idx_pedido_data ON infrastore.pedido(data_hora);
-- Acelera o processo de busca e login de clientes pelo e-mail
CREATE INDEX idx_cliente_email ON infrastore.cliente(email_cliente);
-- Otimiza as buscas e JOINS filtrados por categoria
CREATE INDEX idx_produto_categoria ON infrastore.produto(id_categoria_produto_fk);
```

9 - RBAC + Views

CRIAÇÃO DE USUÁRIOS

```
CREATE USER gerente_user WITH PASSWORD '123';
CREATE USER operador_logistica_user WITH PASSWORD '123';
CREATE USER analista_user WITH PASSWORD '123';
```

CRIAÇÃO DE ROLES

```
CREATE ROLE Gerente_role;
CREATE ROLE operador_logistica_role;
CREATE ROLE analista_role;
```

CONCEDENDO PERMISSÕES AS ROLES

```
GRANT Gerente_role TO gerente_user;
GRANT analista_role TO analista_user;
GRANT operador_logistica_role TO operador_logistica_user;

GRANT USAGE ON SCHEMA infrastore TO Gerente_role;
GRANT SELECT ON ALL TABLES IN SCHEMA infrastore TO Gerente_role;
GRANT UPDATE ON infrastore.produto TO Gerente_role;

GRANT USAGE ON SCHEMA infrastore TO operador_logistica_role;
GRANT SELECT, UPDATE ON infrastore.pedido TO operador_logistica_role;
GRANT SELECT, UPDATE ON infrastore.produto TO operador_logistica_role;
GRANT SELECT ON infrastore.transportadora, infrastore.endereco, infrastore.cep, infrastore.cidade
TO operador_logistica_role;

GRANT USAGE ON SCHEMA infrastore TO analista_role;
GRANT SELECT ON ALL TABLES IN SCHEMA infrastore TO analista_role;
```

CRIANDO VIEWS PARA AS ROLES

1. Criação da View

```
➤ CREATE OR REPLACE VIEW College_Restricted_Logistica_View AS
  SELECT
    p.numero_pedido,
    p.status_pedido,
    p.data_hora,
    c.email_cliente,
    cep.logradouro,
    e.numero_casa,
    t.razao_social_transportadora
  FROM infrastore.pedido p
  JOIN infrastore.cliente c ON p.id_cliente_fk = c.id_cliente
  JOIN infrastore.endereco e ON p.id_endereco_fk = e.id_endereco
  JOIN infrastore.cep cep ON e.id_cep_fk = cep.id_cep
  JOIN infrastore.transportadora t ON p.cnpj_transportadora_fk = t.cnpj_transportadora;
➤ REVOKE ALL PRIVILEGES ON
  infrastore.pessoa_fisica,
  infrastore.pessoa_juridica,
  infrastore.pagamento
  FROM operador_logistica_role;
GRANT SELECT ON College_Restricted_Logistica_View TO operador_logistica_role;
```

2. Criação da View

```
● CREATE OR REPLACE FUNCTION infrastore.tg_atualizar_gerente_produtos()  
  RETURNS TRIGGER AS $$  
  BEGIN  
    UPDATE infrastore.produto  
    SET preco_base = NEW.preco_base,  
        quant_produto = NEW.quant_produto,  
        nome_produto = NEW.nome_produto,  
        descricao_produto = NEW.descricao_produto  
    WHERE cod_produto = OLD.cod_produto;  
  
    RETURN NEW;  
  END;  
  $$ LANGUAGE plpgsql;  
  
● CREATE TRIGGER trg_gerente_update_view  
  INSTEAD OF UPDATE ON infrastore.vw_gerente_gestao_produtos  
  FOR EACH ROW  
  EXECUTE FUNCTION infrastore.tg_atualizar_gerente_produtos();  
  
● CREATE OR REPLACE VIEW infrastore.vw_gerente_gestao_produtos AS  
  SELECT  
    p.cod_produto,  
    p.nome_produto,  
    p.preco_base,  
    p.quant_produto,  
    c.nome_categoria,  
    p.descricao_produto  
  FROM infrastore.produto p  
  JOIN infrastore.categoria c ON p.id_categoria_produto_fk = c.id_categoria_produto;  
  
  GRANT SELECT, UPDATE ON infrastore.vw_gerente_gestao_produtos TO Gerente_role;
```

10 - Criptografia

CÓDIGO

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;  
ALTER TABLE infrastore.funcionario ADD COLUMN senha_funcionario text;  
  
UPDATE infrastore.funcionario SET senha_funcionario = crypt('mudar123', gen_salt('bf'));  
  
ALTER TABLE infrastore.funcionario ALTER COLUMN senha_funcionario SET NOT NULL;
```


INSERT DE FUNCIONÁRIOS

```
UPDATE infrastore.funcionario
SET senha_funcionario = crypt('senhaAna123', gen_salt('bf')) WHERE matricula_funcionario = 'FUN001';

UPDATE infrastore.funcionario
SET senha_funcionario = crypt('bruno@pass', gen_salt('bf')) WHERE matricula_funcionario = 'FUN002';

UPDATE infrastore.funcionario
SET senha_funcionario = crypt('carlaAcesso', gen_salt('bf')) WHERE matricula_funcionario = 'FUN003';

UPDATE infrastore.funcionario
SET senha_funcionario = crypt('diegoMudar9', gen_salt('bf')) WHERE matricula_funcionario = 'FUN004';

UPDATE infrastore.funcionario
SET senha_funcionario = crypt('eva@2026', gen_salt('bf')) WHERE matricula_funcionario = 'FUN005';
```

11 - Análises Críticas das Decisões de Modelagem (Parte 2)

Para aproximar o banco de dados InfraStore de um cenário corporativo real, foram implementadas camadas adicionais de otimização, controle de acesso e criptografia, garantindo a resiliência operacional do sistema.

11.1. Otimização de Consultas (Índices)

- **Recurso Utilizado:** Criação de Índices (B-Tree padrão do PostgreSQL) utilizando o comando **CREATE INDEX**.
- **Decisões de Implementação:** A decisão de onde aplicar os índices baseou-se nos gargalos mais comuns de sistemas de e-commerce. Não indexamos todas as colunas para evitar lentidão nas inserções (inserts). Os índices foram aplicados estrategicamente em:
 - **data_hora (Tabela Pedido):** Para acelerar a geração de relatórios financeiros mensais baseados em recortes de tempo.
 - **email_cliente (Tabela Cliente):** Para otimizar o tempo de resposta no processo de login e autenticação na plataforma.
 - **id_categoria_produto_fk (Tabela Produto):** Para agilizar as junções (JOINS) na montagem de catálogos filtrados por setor.

11.2. Controle de Acesso Baseado em Papéis (RBAC)

- **Recurso Utilizado:** Criação de papéis no SGBD (**CREATE ROLE**), atrelados a usuários virtuais, utilizando os comandos de controle de acesso **GRANT** e **REVOKE**.
- **Decisões de Implementação:** Adotou-se o "Princípio do Menor Privilégio". Em vez de utilizar um único superusuário para a aplicação, o domínio foi dividido em três perfis operacionais rígidos:
 - **Gerente:** Possui privilégios de leitura e atualização (**UPDATE**) sobre o catálogo de produtos e configurações estruturais da loja.
 - **Analista:** Perfil estritamente de leitura (**SELECT**). Garante a extração de dados para relatórios de Business Intelligence sem o risco de alterações acidentais no banco.
 - **Operador de Logística:** Perfil operacional com permissão de atualização restrita ao controle de despacho (**status_pedido**).

11.3. Segurança e Controle de Visibilidade (Views Restritivas)

- **Recurso Utilizado:** Criação de Views Virtuais (**CREATE VIEW**) combinada com a revogação de acessos diretos às tabelas base.
- **Decisões de Implementação:** O setor de expedição/logística precisa despachar pacotes, mas não deve ter acesso a informações confidenciais ou valores financeiros por questões de privacidade (LGPD). A decisão arquitetural foi revogar o acesso do perfil **operador_logistica_role** às tabelas **pessoa_fisica** e **pagamento**. O acesso aos dados de envio foi liberado exclusivamente através da visão **College_Restricted_Logistica_View**, que oculta o CPF e o total gasto, expondo apenas o destino físico e a transportadora alocada.

11.4. Criptografia de Dados Sensíveis e Autenticação

- **Recurso Utilizado:** Extensão nativa **pgcrypto** do PostgreSQL, utilizando a função **crypt()** combinada com a geração de salt aleatório **gen_salt('bf')**.
- **Decisões de Implementação:** O armazenamento de credenciais em texto limpo é uma falha crítica de segurança. Foi adicionada a coluna **senha_funcionario**, e decidiu-se implementar a criptografia utilizando o algoritmo de hashing Blowfish (**bf** / **bcrypt**). A escolha deste algoritmo justifica-se pelo uso de um Salt (tempero criptográfico aleatório) para cada linha, o que garante que senhas iguais gerem hashes diferentes no banco, mitigando ataques de força bruta e garantindo a resiliência do sistema em caso de vazamento de dados.

12 - Links

Link do LucidChart (MER): [LucidChart](#)

Link do Repositório: [InfraStore](#)